Lock-Based vs. Lock-Free Designs

Unit 3 – Synchronization and Concurrency Control Design

Prashant Jadav

DEPSTAR – CHARUSAT

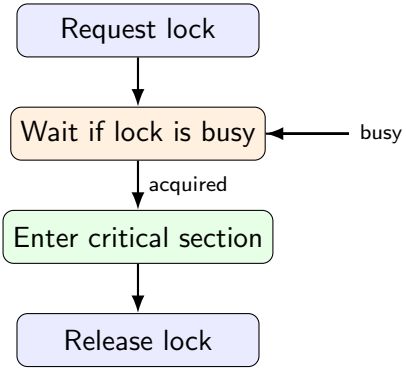
Why Synchronization Design Matters

- Concurrent threads may access and modify shared data simultaneously.
- Synchronization must prevent data races and preserve data-structure invariants.
- **Lock-based designs** use locks to protect critical sections.
- **Lock-free designs** use atomic operations without relying on lock ownership for system-wide progress.
- The choice affects correctness, throughput, latency, fairness, and complexity.

Important distinction

Lock-free does not mean no synchronization; it means progress does not depend on a thread holding a mutual-exclusion lock.

Lock-Based Design: Basic Idea



- Mutual exclusion allows only one thread at a time in the protected region.
- Waiting threads may block, spin, or be descheduled.
- Examples: mutexes, spinlocks, read-write locks, monitors.

Lock-Based Example: Protecting a Counter

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void increment(void) {
    pthread_mutex_lock(&lock);
    counter = counter + 1;
    pthread_mutex_unlock(&lock);
}
```

- The mutex protects the critical section.
- Concurrent increments are serialized, preventing lost updates.
- Every execution path must release the lock; otherwise, other threads may wait indefinitely.

Lock-Based Design: Strengths and Risks

Strengths

- Familiar model
- Easier to reason about multi-step invariants
- Often effective for coarse-grained critical sections

Risks

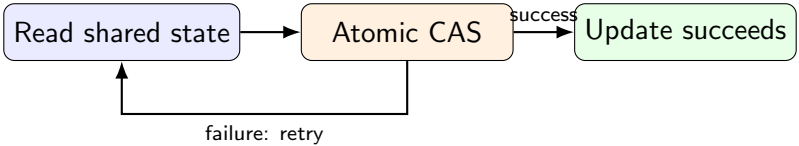
- Deadlock from inconsistent lock ordering
- Starvation or priority inversion
- Lock contention and context-switch overhead
- A delayed lock holder can block others

Design question

Can a simple lock protect the state correctly, or is non-blocking progress a genuine requirement?

Lock-Free Design: Basic Idea

- Uses atomic primitives such as compare-and-swap (CAS), exchange, or fetch-add.
- Threads attempt atomic state transitions instead of acquiring a mutual-exclusion lock.
- If one thread is delayed, other threads can still make system-wide progress.
- Failed attempts may cause a thread to reread state and retry.



Progress guarantee

In a lock-free algorithm, at least one concurrent operation completes in a finite number of steps; an individual thread may still starve.

Lock-Free Example: Atomic Counter

```c
#include <stdatomic.h>
atomic_int counter = 0;

void increment(void) {
    atomic_fetch_add(&counter, 1);
}
```

- Atomic read-modify-write prevents concurrent increments from overwriting each other.
- Whether the operation is implemented without locks depends on the platform and atomic type.
- An atomic variable alone does not make a larger multi-step algorithm correct.

Compare-and-Swap (CAS)

Concept

CAS atomically compares a memory location with an expected value. If equal, it replaces the value with a desired value; otherwise, it reports failure.

$$\mathrm{CAS}(\mathit{address},\ \mathit{expected},\ \mathit{desired})$$

1. Read the current state.
2. Compute the desired update.
3. Attempt CAS using the previously observed value.
4. If another thread changed the state first, reread and retry.

Caution

CAS makes one state transition atomic; it does not automatically make an entire algorithm correct.

Lock-Based vs. Lock-Free: Comparison

Dimension	Lock-based	Lock-free
Coordination	Lock ownership and critical sections	Atomic state transitions and retries
Blocking	A thread may wait for lock owner	No lock owner is required for system-wide progress
Deadlock	Possible if locks are misused	Lock-induced deadlock avoided; starvation/livelock may remain
Contention	Waiting, spinning, or blocking	CAS failures and repeated retries
Complexity	Usually simpler	Usually harder to design and verify
Performance	Often good for simple critical sections	May help under some contention patterns; not always faster

Progress Guarantees: Blocking, Lock-Free, Wait-Free

Guarantee	Meaning
Blocking	A thread may wait indefinitely for another thread to release a resource.
Lock-free	System-wide progress: some operation completes in finite steps, even if an individual thread may starve.
Wait-free	Every operation completes within a bounded number of its own steps, regardless of other threads.

Relationship

Wait-free is stronger than lock-free. Lock-free does not necessarily imply starvation-free.

Performance: What Should Be Measured?

- **Throughput**: completed operations per unit time.
- **Latency and tail latency**: time taken by individual operations, including slow cases.
- **CPU utilization**: useful work versus spinning and retries.
- **Scalability**: performance as cores and concurrency increase.
- **Fairness**: whether threads receive reasonable opportunities to complete.

Do not assume

Lock-free is not automatically faster. CAS retries, cache-line bouncing, and memory reclamation can outweigh benefits.

Memory Reclamation in Lock-Free Structures

- In lock-free linked lists, queues, and stacks, one thread may remove a node while another still holds its pointer.
- Freeing a node too early can cause use-after-free errors or ABA-related bugs.
- Common techniques include hazard pointers, epoch-based reclamation, and read-copy-update (RCU), where appropriate.
- Correct atomic updates are not enough; object lifetime and reclamation must also be correct.

Practical implication

Lock-free dynamic data structures often require more careful memory-management reasoning than lock-based counterparts.

Choosing Between the Designs

Prefer lock-based when...	**Consider lock-free when...**
The critical section is short and simple.	Blocking is unacceptable for a key requirement.
The invariant spans multiple operations.	A well-tested lock-free algorithm fits the workload.
Maintainability and straightforward reasoning dominate.	Measured contention justifies extra complexity.
Mutex support is robust on the platform.	Required atomics are efficient on the target platform.

Engineering rule

Start with the simplest correct design, measure it, and adopt lock-free only when requirements and evidence justify it.

Scenario-Based Analysis

A multi-core server maintains a shared request counter and a linked queue of pending jobs.

1. Would a mutex-protected counter or an atomic counter be simpler? Why?
2. What can happen if a thread holding a mutex is descheduled?
3. Under heavy contention, what costs might a CAS loop introduce?
4. Why does a lock-free queue need safe memory reclamation?
5. Which metrics should be collected before choosing a design?

Expected analysis

Discuss correctness, progress guarantees, contention, latency, complexity, and workload.

Exam-Oriented Questions

1. Differentiate lock-based and lock-free synchronization.
2. Explain CAS and its role in a lock-free algorithm.
3. Distinguish blocking, lock-free, and wait-free progress guarantees.
4. Why does lock-free not imply starvation-free?
5. Compare lock contention with CAS retry overhead.
6. Explain why memory reclamation is challenging in lock-free data structures.
7. Justify a synchronization design for a shared counter and concurrent queue.

Summary

- Lock-based designs use mutual exclusion to protect shared state.
- Lock-free designs use atomic operations to permit system-wide progress without relying on lock ownership.
- CAS is a common building block but does not guarantee whole-algorithm correctness.
- Lock-free is not the same as wait-free, starvation-free, or always faster.
- Evaluate throughput, latency, scalability, CPU cost, and fairness.
- Memory reclamation is a major concern for lock-free dynamic data structures.

Key takeaway

Choose synchronization by balancing correctness, progress guarantees, measured performance, and maintainability.

References

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, *Operating System Concepts*, 10th Edition.
- William Stallings, *Operating Systems: Internals and Design Principles*, 9th Edition.
- Maurice Herlihy and Nir Shavit, *The Art of Multiprocessor Programming*.
- ISO C documentation on atomic operations and memory ordering.